用户界面设计文档

1. 文档概述

1.1 编写目的

本文档描述"面向 CPU+XPU 异构系统的智能调度算法及中间件原型"面向**管理人员**的用户界面设计。中间件本体为常驻守护进程，没有图形界面；管理人员通过**命令行界面（CLI）**完成查询系统状态、启停服务、切换调度模式三类运维操作。本文档为 `schedctl` 客户端工具的实现提供依据。

1.2 读者对象

项目开发团队成员、指导教师、评审人员。

1.3 术语定义

术语	说明
CLI	Command Line Interface，命令行界面
子命令	位于 `schedctl` 之后、表示一类操作的动词（如 `status`）
动作	位于子命令之后、表示具体操作的词（如 `service start`）
参数	位置参数，如 `mode auto` 中的 `auto`
选项	以 `-` 或 `--` 开头的可选项，如 `--json`
退出码	进程返回值，用于脚本判定（0 成功，非 0 失败）
UDS	Unix Domain Socket，与守护进程通信的通道
模式	调度模式 auto / static / off

1.4 参考资料

- 《软件需求规格说明书.md》
- 《软件体系结构设计文档.md》
- 《初步软件需求.md》

2. 系统概述

2.1 软件目标

构建可集成于 openEuler 的轻量化调度中间件，通过 DQN 推理为 LLM 推理任务动态分配 CPU/NPU 资源、批处理大小及频率档位，并为管理人员提供可运维性。

2.2 主要功能

- 面向推理引擎：请求推理调度（自动触发采集系统状态、调整处理器频率）
- 面向管理人员：**查询系统状态、启停服务、切换调度模式**

2.3 目标用户

用户	使用方式	关注点
管理人员（运维）	`schedctl` 命令行	当前状态、服务可用性、调度模式

2.4 运行环境

- openEuler 24.03 及以上，终端环境（bash）
- `schedctl` 与守护进程 `schedd` 同机部署，通过 UDS 通信
- 服务启停委托 systemd（`systemctl`）

3. 设计目标和原则

3.1 设计目标

- 覆盖管理人员全部三个用例：查询系统状态、启停服务、切换调度模式。
- 命令结构直观、可发现（discoverable），管理人员无需查手册即可入门。
- 输出既能被人阅读，也能被脚本消费（提供 `--json`）。
- 危险操作有保护（关闭调度需二次确认）。

3.2 设计原则

原则	在 CLI 中的落实
直观性	子命令用运维领域动词 status / mode / service，含义一目了然
易操作性	常见操作单行命令完成，无多余跳转；`schedctl status` 即查即显
一致性	统一 `schedctl <子命令> [动作] [参数] [选项]` 结构；统一退出码语义
反应性	查询类命令即时返回；`--watch` 周期刷新并高亮变化字段
容错性	参数非法时给出用法提示；`mode off` 与 `service stop` 二次确认
人性化	无参数执行即打印帮助；错误信息包含"下一步怎么办"的提示

4. 用户界面清单

编号	界面	命令行形式	职责	对应用例
UI1	主命令界面	`schedctl`	展示系统主要功能与用法，作为入口与总览	—（导航）
UI2	状态查询界面	`schedctl status`	展示处理器状态与调度模式	UC4 查询系统状态
UI3	模式切换界面	`schedctl mode [<模式>]`	查询/切换调度模式	UC6 切换调度模式
UI4	服务管理界面	`schedctl service <动作>`	启停/重启/查看守护进程	UC5 启停服务

UI1 为主界面：管理人员首次使用或误用时都会落到该界面，并从该界面进入其余界面。

5. 界面元素与操作设计

5.1 UI1 主命令界面（`schedctl`）

静态元素

元素	类型	说明
程序名	文本	`schedctl`
子命令列表	文本（菜单）	`status` / `mode` / `service` / `version` / `help`
`-h, --help`	选项	显示完整帮助
`-v, --version`	选项	显示版本号
用法示例	文本	3~5 条常见用法

动态元素

元素	来源	说明
版本号	编译期宏 `SCHEDCTL_VERSION`	与 schedd 版本对应
子命令摘要	命令注册表	依据编译开关动态生成

操作清单

操作	触发条件	结果
显示帮助	无参数执行 / `-h`	打印帮助文本，退出码 0
显示版本	`-v` / `--version`	打印版本，退出码 0
进入子界面	输入已注册子命令	跳转至 UI2 / UI3 / UI4
报错	输入未知子命令	打印 `unknown command` + 用法提示，退出码 2

5.2 UI2 状态查询界面（`schedctl status`）

静态元素

元素	类型	说明
子命令	文本	`status`
`--json`	选项	以 JSON 输出，便于脚本消费
`--watch`	选项	周期刷新模式
`-i <秒>`	选项（带参）	刷新间隔，默认 1

动态元素

元素	含义
调度模式	当前模式 auto / static / off
CPU 利用率	由 `/proc/stat` 差值计算，百分比
NPU 三核负载	三个 NPU 核的负载，百分比
当前频率	CPU / NPU 当前工作频率（MHz）
governor	当前调频策略
守护进程运行时长	schedd 已运行时间
最近决策时间	最近一次 DQN 决策的时间戳与耗时
连接状态	与 schedd 的 UDS 连接是否正常

操作清单

操作	触发条件	结果
单次查询	默认	连接 schedd，拉取一次状态并格式化打印
周期查询	`--watch`	按 `-i` 间隔重复查询并就地刷新
机器可读输出	`--json`	输出单个 JSON 对象
连接失败	schedd 未运行	打印错误 + 提示 `schedctl service start`，退出码 3

5.3 UI3 模式切换界面（`schedctl mode`）

静态元素

元素	类型	说明
子命令	文本	`mode`
位置参数	枚举	`<auto
`--dry-run`	选项	仅校验不生效
`-y, --yes`	选项	跳过确认

动态元素

元素	含义
当前模式	未带参数时显示的模式值
切换结果	成功/失败及生效后的模式
确认提示	切换到 off 时的危险操作确认

操作清单

操作	触发条件	结果
查询模式	`mode` 无参数	显示当前模式，退出码 0
切换模式	`mode auto` / `mode static`	下发切换并回显结果
关闭调度	`mode off`	二次确认后切换；未确认则取消
试运行	`--dry-run`	只校验参数与权限，不实际下发

5.4 UI4 服务管理界面（`schedctl service`）

静态元素

元素	类型	说明
子命令	文本	`service`
动作	枚举	`<start
`-y, --yes`	选项	跳过 stop 的确认

动态元素

元素	含义
服务状态	active / inactive / failed
进程号	schedd 的 PID
最近一次退出码	上一次退出状态

操作清单

操作	触发条件	结果
启动	`service start`	调用 `systemctl start schedd`
停止	`service stop`	二次确认后调用 `systemctl stop schedd`
重启	`service restart`	调用 `systemctl restart schedd`
查看	`service status`	读取服务状态并展示

6. 界面跳转关系

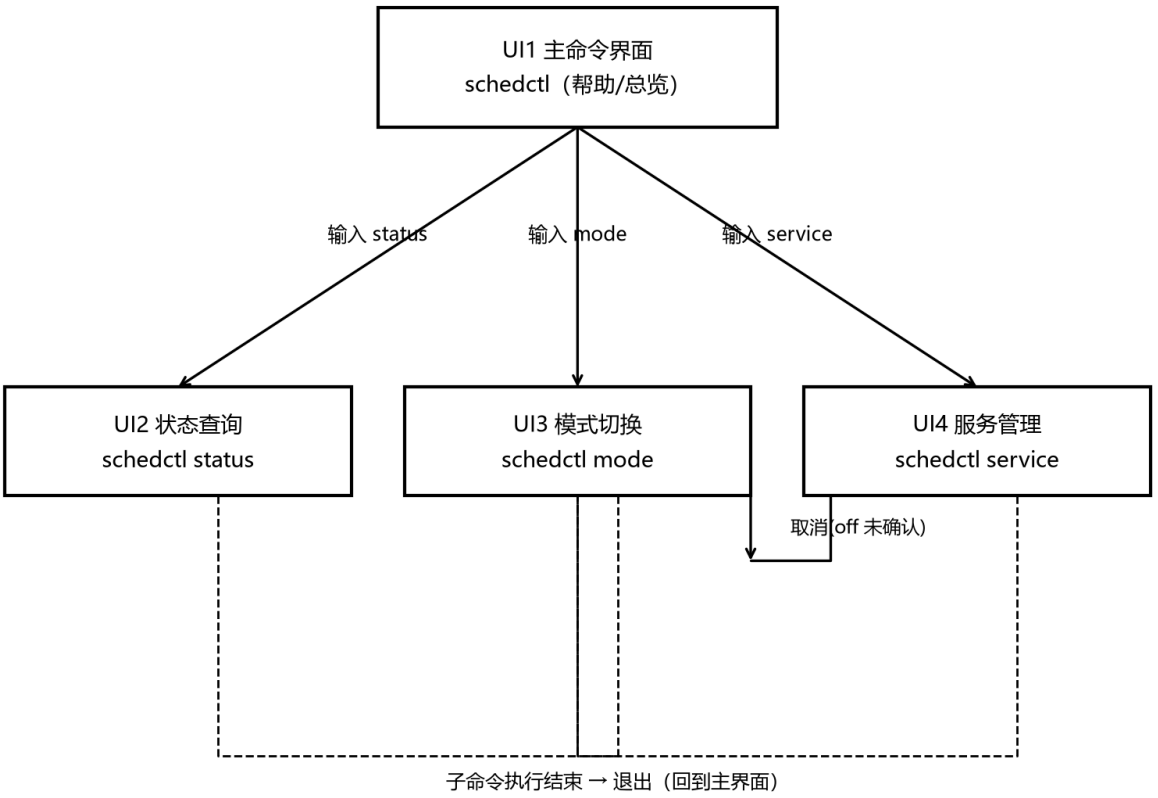
6.1 主界面说明

主界面为 **UI1（`schedctl` 无参数/帮助）**：它展示系统主要功能（三个运维能力与版本/帮助），管理人员从它进入其余任意界面；每个子界面执行完成后即退出并回到提示符（等价于返回主界面）。

6.2 跳转关系

源界面	触发事件/条件	目标界面
UI1	输入 `status`	UI2 状态查询
UI1	输入 `mode`	UI3 模式切换
UI1	输入 `service`	UI4 服务管理
UI1	输入 `help` / 无参数	UI1（自刷新帮助）
UI2	命令结束 / `Ctrl+C`（watch）	UI1（退出）
UI3	查询/切换结束	UI1（退出）
UI3	选择 `off` 且未确认	UI3（取消，留在本界面）
UI4	动作结束	UI1（退出）

跳转图如下：



注：三个子界面互不跳转，均回归主界面

7. 精化设计说明

- **查漏补缺**：为配合 `--watch`，UI2 补充了"刷新间隔 `-i`"元素；为危险操作保护，UI3 补充 `-y` 与确认提示；为脚本化，UI2/UI3/UI4 统一提供退出码。

- **建立跳转**：将每个命令的"执行完成"显式建模为返回主界面；将 `mode off` 的确认子流程建模为 UI3 内部自跳转。
- **合并/拆分**：三类运维操作拆为三个子命令界面，避免单命令多重语义；版本/帮助合并进 UI1，不再单列界面。
- **布局与一致性**：统一 `用法 / 选项 / 示例` 三段式帮助；统一输出字段顺序（模式 → CPU → NPU → 频率 → governor）；错误信息统一以 `error:` 前缀并附修复提示。
- **美化**：终端输出对齐列宽；`--watch` 模式下变化字段高亮；危险确认使用醒目提示。
- **模型同步**：本节的界面清单、元素与操作已同步反映到第 5 章与第 9 章模型。

8. 界面原型说明

原型以终端文本形式呈现（无需图形控件）。典型交互如下。

UI1 帮助

```
$ schedctl
schedctl - CPU+XPU 异构调度中间件管理工具 v0.1.0

用法:
  schedctl <子命令> [动作] [参数] [选项]

子命令:
  status     查询处理器状态与调度模式
  mode       查询或切换调度模式 (auto|static|off)
  service    启停/重启/查看 schedd 服务
  version    显示版本
  help       显示本帮助

示例:
  schedctl status --watch
  schedctl mode auto
  schedctl service restart
```

UI2 状态查询

```
$ schedctl status
模式         : auto (DQN)
CPU 利用率   :  42.7%
NPU 负载     :  core0  38%   core1  41%   core2  35%
频率         :  CPU 1800MHz   NPU 1000MHz
governor     :  performance
运行时长     :  3d 04:12:07
最近决策     :  12ms 前 (耗时 1.8ms)
连接         :  OK (UDS)
```

UI3 模式切换

```
$ schedctl mode off
警告：关闭自动调度将退化为纯监控，推理任务不再获得调度建议。
确认切换到 off 模式？[y/N] y
模式已切换：auto -> off
```

UI4 服务管理

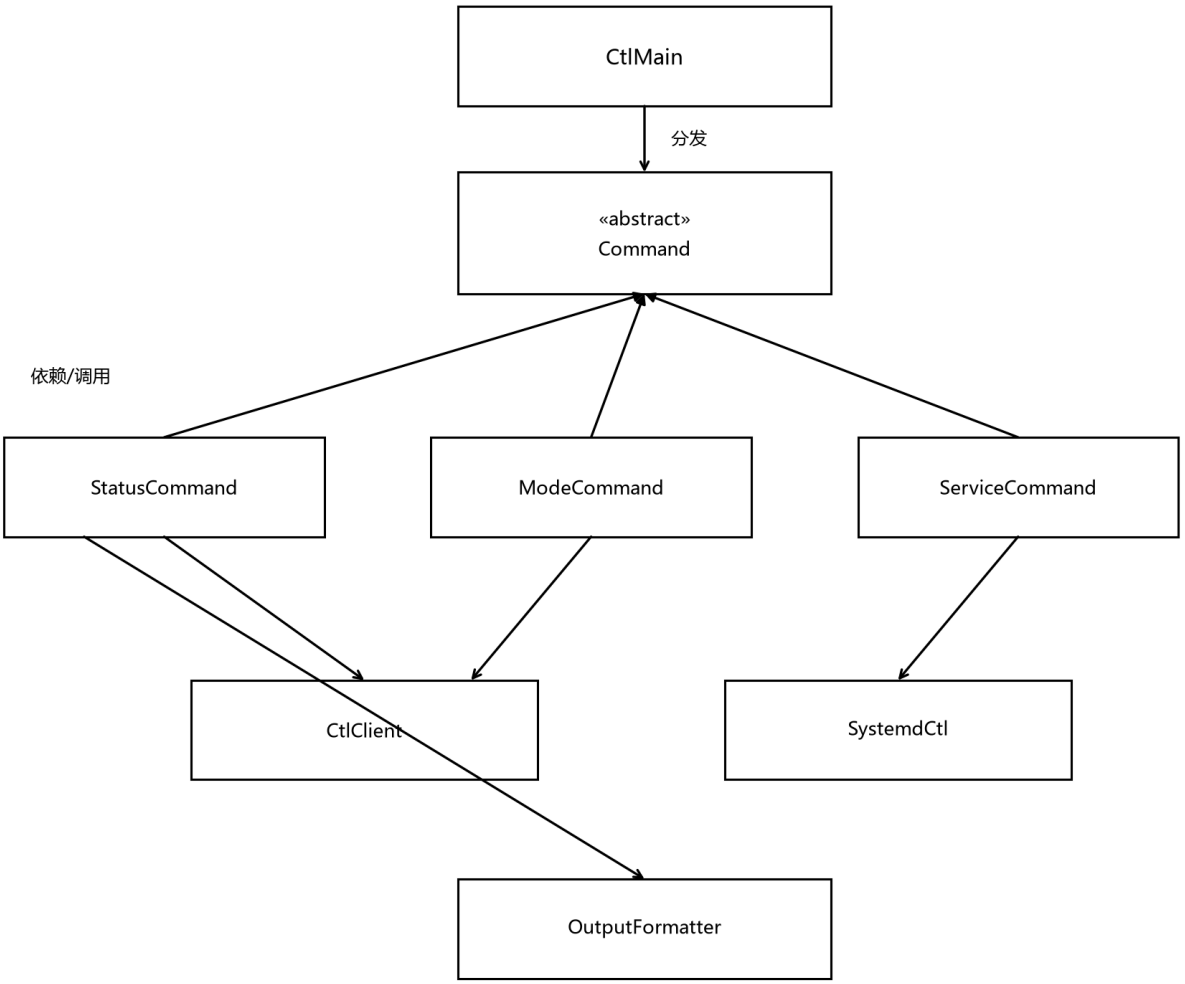
```
$ schedctl service status
schedd.service - 智能调度中间件守护进程
   Active: active (running) since Mon 2026-10-13 08:00:00 CST
   PID   : 2145
```

9. 界面 UML 模型

9.1 界面类图

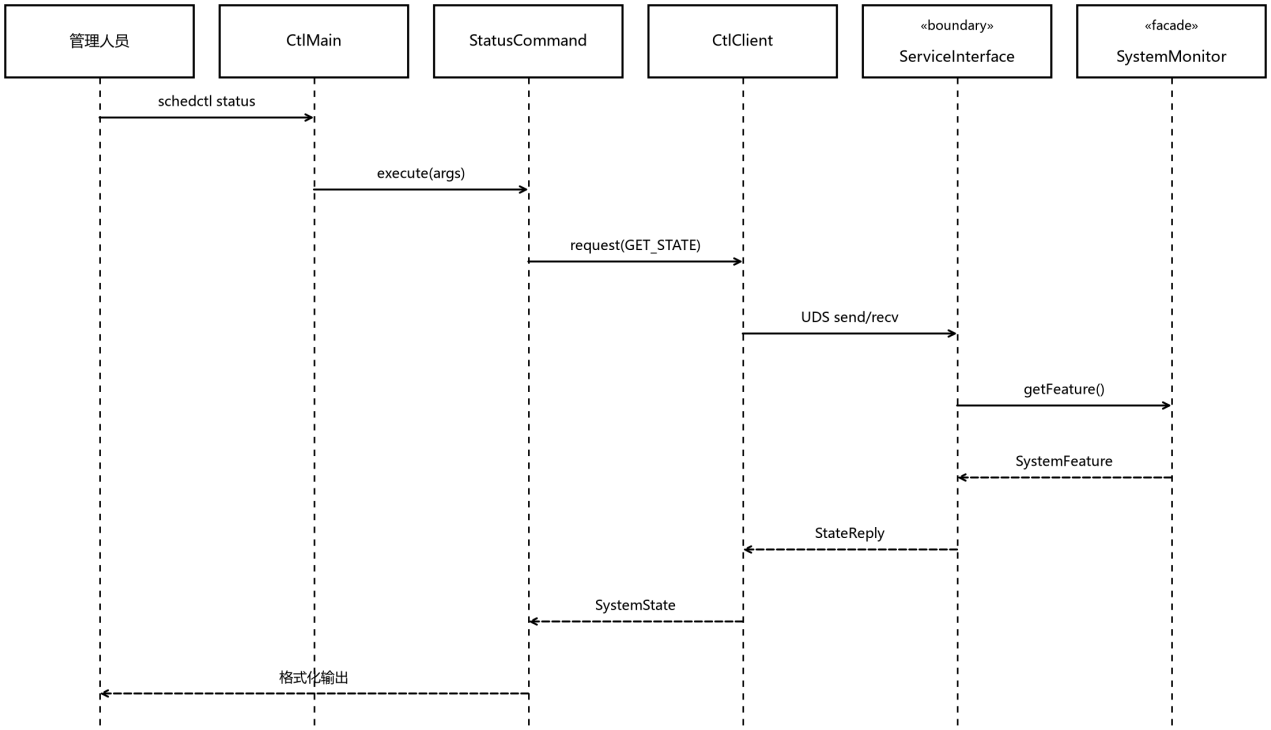
界面侧由"主控 + 命令 + 通信/系统委托 + 格式化"四类对象组成：

界面类	类型	职责
`CtlMain`	主控	解析 argv、分发给命令、统一退出码
`Command`（抽象）	命令接口	声明 `execute(args)`
`StatusCommand`	命令	查询并展示状态
`ModeCommand`	命令	查询/切换模式
`ServiceCommand`	命令	委托 systemd 管理服务
`CtlClient`	边界/通信	通过 UDS 与 schedd 交互
`SystemdCtl`	边界/委托	封装 `systemctl` 调用
`OutputFormatter`	格式化	表格 / JSON 输出



9.2 界面顺序图（`schedctl status`）

管理人员输入 `schedctl status` → `CtlMain` 解析并分发 → `StatusCommand` 经 `CtlClient` 与守护进程交互 → 结果经 `OutputFormatter` 打印。



10. 评审意见

#	评审方面	结论
①	需求覆盖	通过：UC4/UC5/UC6 三个管理人员用例均有对应界面（UI2/UI4/UI3），UC1~UC3 由中间件自动完成、无界面需求
②	操作习惯	通过：子命令采用运维惯用动词，符合 Linux CLI 习惯
③	风格一致	通过：统一 子命令 [动作] [参数] [选项] 结构与退出码语义
④	美观直观	通过：终端对齐列宽、三段式帮助、危险操作高亮确认
⑤	布局与跳转	通过：主界面 UI1 进入其余界面，子界面结束后回归；跳转与用例交互逻辑一致
⑥	原型与模型一致	通过：第 8 章原型与第 9 章类图/顺序图一致
⑦	元素间一致	通过：静态元素（选项）与动态元素（输出字段）一一对应，无孤立元素

评审结论：七项全部通过，进入软件详细设计阶段。